Generative AI — Complete Course Notes

A clean, explained version of your class notes, topic by topic.

---

1. Introduction to Generative AI (Day 1)

Traditional AI vs Generative AI

	Traditional AI / ML	Generative AI
What it does	Classifies, predicts, or scores based on patterns it learned	*Creates* new content — text, images, code, audio, video
Training data size	Small/limited (hundreds to thousands of examples)	Massive (internet-scale — millions/billions of examples)
Model size	Small number of parameters	Millions to billions (even trillions) of parameters
Example	A model that predicts house prices, or classifies spam vs not-spam	ChatGPT writing an essay, Midjourney drawing an image

The core building block behind modern Generative AI is the **LLM (Large Language Model)** — a neural network trained on huge amounts of text so it learns the statistical patterns of language well enough to generate new, coherent text.

ChatGPT as the first mainstream application ChatGPT (built on GPT models) was the first widely-used generative AI application. Importantly, the *first* version was **text-to-text only** — you typed text in, you got text out. It could not generate images.

LLM vs Multimodal

- **LLM (pure)**: only handles text → text (e.g., understanding and generating language).
- **Multimodal model**: can move *across* formats — text→image, text→video, text→audio, image→text, etc.
- Multimodal models are often called **State-of-the-Art (SOTA)** models because they represent the current frontier of capability.
- You'll also hear "**cognitive AI**" used loosely to describe this broader, multi-sense capability (understanding across text, vision, audio like a human would).

The major LLM families (as of your notes)

- **OpenAI**: GPT-2 → GPT-3 → GPT-3.5 → GPT-4 → GPT-4o → GPT-5 → GPT-5.x (naming continues to evolve)
- **Google**: Gemini 1 → Gemini 1 Pro/Flash → Gemini 2 → Gemini 2 Flash/Flash Thinking → Gemini 3
- **Anthropic**: Claude — Sonnet, Opus, Haiku, and Fable model lines, each at different capability/speed/cost tiers
- **Meta**: LLaMA 1 → 2 → 3 → 4 (Maverick & Scout variants). Meta's models are **open-weight**, so they're mainly used for **fine-tuning** rather than being used directly like a hosted product.

> **Note:** Model version numbers and names change frequently — treat any specific version number as a snapshot in time, not a fixed fact.

---

2. How Generative AI Works (7th)

Two foundational architectures explain *how* generative models learn to create realistic content: **GANs** and **VAEs**.

GAN — Generative Adversarial Network A GAN has two competing neural networks:

- **Generator**: tries to create fake data (e.g., an image) that looks real.
- **Discriminator**: acts like a judge/classifier — it looks at an image and decides "real" or "fake."

Adversarial training = the two networks are trained against each other in a loop:

1. Generator creates a fake image.
2. Discriminator tries to catch that it's fake.
3. Generator adjusts to fool the discriminator better next time.
4. Repeat thousands of times.

Over time, the Generator gets so good that the Discriminator can no longer reliably tell real from fake — at that point, the Generator produces convincingly realistic content.

VAE — Variational Autoencoder A different generative approach built from an **Encoder** and a **Decoder**:

- **Encoder**: compresses input (e.g., an image or user's prompt) into a compact internal representation.
- **Decoder**: expands that internal representation back out into an output.

In an LLM context, you can think of it loosely as:

> User input → **Encoder** (understands/compresses meaning) → internal representation → **Decoder** (generates the output) → response

Types of Gen AI models

- **Text-based models**: generate *only* text (no image/video/audio). Example strength: Claude.
- **Multimodal models**: can generate across image, video, audio *and* text. These are the SOTA models (e.g., Gemini, GPT-4o+).

How to build a Gen AI model — the 5 ingredients

1. **Data** — huge datasets (internet text, books, websites). Someone has to supply/curate this.
2. **Compute** — regular CPUs cannot handle this scale; you need high-end **GPU infrastructure**.
3. **Model architecture** — the underlying design (e.g., CNN for images, RNN for older sequence models, Transformers for modern LLMs).
4. **Training algorithm** — e.g., GAN-style adversarial training, or the Transformer training process.
5. **Difficulty** — building a Gen AI model from scratch is far harder than building a traditional ML model; it's not something you do "manually" the way you might train a small classifier.

---

3. Tokenization, Embeddings & Attention (8th)

Tokenization: how text becomes something a model can process

Before any text reaches a model, it must be broken into **tokens** (pieces of text). Three common tokenization strategies:

- **BPE (Byte Pair Encoding)**: Starts with individual characters, then repeatedly merges the *most frequent* pair of characters/symbols together, building up a vocabulary of common sub-words step by step.
- **WordPiece**: Similar idea, but instead of picking purely by frequency, it picks merges that **maximize the likelihood** of the training data under the resulting vocabulary (a more probability-driven choice than pure frequency).
- **SentencePiece**: Works directly on raw text *without* assuming words are separated by spaces — useful for languages like Japanese or Chinese where there's no clear space-separated word boundary. Space itself is treated as just another symbol.

Why embeddings matter

An **embedding** turns a token (or a whole chunk of text) into a list of numbers (a vector) that captures its *meaning* — similar meanings end up as vectors that are close together in this "meaning space." This is the foundation of **RAG** (see Topic 12) and of computing similarity between texts.

Attention: how a model figures out *which* words matter to each other

Attention solves a real problem: **many words have multiple meanings**, and a model needs context to disambiguate them.

- **Self-Attention**: for a *single* word with multiple possible meanings, the model looks at surrounding words in the same sentence to figure out which meaning applies. Example: "bank" means something different in "river bank" vs "money bank" — self-attention lets the model use context to decide.
- **Multi-Head Attention**: extends this — when there are **multiple words**, each potentially ambiguous in its own way, the model runs several attention "heads" in parallel, each focusing on different types of relationships between words, then combines the results. This is what lets Transformers understand complex sentences without collapsing into ambiguity (hallucination risk goes up when this isn't handled well).

Q, K, V — the mechanics behind attention

- **Query (Q)** — represents "what am I looking for?" from the current word's perspective.
- **Key (K)** — represents "what information do I contain?" for every other word.
- **Value (V)** — represents "what information do I actually contribute?" once relevance is established.

The attention pipeline, step by step:

```
Sentence
   ↓
Tokenization
   ↓
Embeddings
   ↓
Create Q, K, V
   ↓
Q · Kᵀ  (compare query against every key)
   ↓
Scale   (keep numbers in a stable range)
   ↓
Softmax (turn scores into probabilities)
   ↓
Weighted sum of V  (blend values according to those probabilities)
   ↓
Attention Output
```

This whole chain — **Attention → Self-Attention → Multi-Head Attention → Positional Encoding → Transformer → Encoder/Decoder → LLM** — is essentially the architecture stack that modern LLMs are built on. (Positional Encoding, not detailed

in your notes, is what tells the model the *order* of words, since attention by itself has no built-in sense of sequence.)

How a whole book gets stored for retrieval (the RAG data pipeline)

Step	What happens	Analogy
1. Chunking	The book/document is split into smaller pieces	Cutting a cake into slices
2. Embedding	Each piece is converted into a numeric vector	Giving each slice GPS coordinates
3. Storing	Vectors are saved into a vector database	Pinning each slice on a map
4. Searching	The user's question is also converted into a vector	Finding your question's spot on the map
5. Retrieving	The database returns the closest-matching pieces	Handing you the nearest slices of cake

Fine-tuning capability by model (as noted in class)

- **ChatGPT (OpenAI)** — cannot be freely fine-tuned by end users on the base flagship models.
- **Claude (Anthropic)** — cannot be fine-tuned by end users.
- **Gemini (Google)** — cannot be freely fine-tuned by end users.
- **LLaMA (Meta)** — **can** be fine-tuned, because it's released as open-weight, so you can download and retrain it yourself.

(Nuance worth knowing: some providers do offer limited, controlled fine-tuning through their paid APIs for specific use cases — but full, open fine-tuning is really only practical with open-weight models like LLaMA, Mistral, etc.)

Context length & knowledge cutoff

- **Context length** = the maximum amount of text (input + output combined) a model can "see" and generate in one go. E.g., an older model with a small context window simply cannot process a very long document in a single request.
- **Knowledge cutoff** = the date up to which a model's training data goes. Ask it about anything after that date, and it either doesn't know or may guess incorrectly (hallucinate).

Data curation & safety

When data is pulled from the internet for training, it needs **guardrails** — filtering to ensure the data comes from safe, credible sources, since the internet contains both reliable and harmful/incorrect content.

---

4. LangChain Fundamentals & LangSmith (9th)

LangChain is a framework that makes it much easier to build applications on top of LLMs. Three key things it gives you:

1. **Universal connection** — LangChain can connect to almost any LLM (OpenAI, Claude, Gemini, etc.) through a consistent interface, so you're not rewriting code for each provider.
2. **Prompt construction** — helper tools to build and manage prompts systematically rather than hardcoding strings.
3. **Chains (via LCEL – LangChain Expression Language)** — the ability to link multiple steps together (e.g., "take input → format prompt → call model → parse output") into a single pipeline.

LLM vs RAG — the hallucination angle (first mention)

- **Pure LLM**: pulls from the general knowledge it learned during training (or general internet knowledge) → higher chance of hallucination when asked about something specific or recent.
- **RAG**: pulls from a **vector database** you control → much lower chance of hallucination, because the model is grounded in retrieved, verified documents.

Practical work (POCs) done in class:

- **POC 1**: LangChain integrated with OpenAI (using a paid API key, e.g., a `gpt-5.x-mini` model) — confirmed working.
- **POC 2**: LangChain integrated with Gemini.

LangSmith A separate but complementary framework used for **tracing, observability, and monitoring** of your LLM application — you can see exactly what prompts were sent, what the model returned, how many tokens were used, and where things might be going wrong.

Projects built:

- Project 1: Chatbot built with OpenAI, monitored via LangSmith.
- Project 2: Chatbot built with Gemini, monitored via LangSmith.

---

5. Prompt Engineering — Parameters & Enterprise Context (10th)

What is Prompt Engineering? The practice of carefully designing the *input* (the prompt) you give an LLM so that it produces the most accurate, relevant, and useful output. Since LLMs are extremely sensitive to how a question is phrased, this has become its own skill.

Why learn it? Because the same underlying model can give a poor answer to a badly-phrased prompt and an excellent answer to a well-structured one — prompt engineering is often the cheapest way to improve output quality without touching the model itself.

Key LLM generation parameters (you'll configure these in almost every API call):

Parameter	What it controls
Model	Which specific LLM/version you're calling
Temperature	Randomness/creativity of output (covered in depth in Topic 6)
Max length (max tokens)	The maximum number of tokens the model is allowed to generate in one response
Stop sequence	A specific string (e.g., "END", "STOP") that tells the model to stop generating immediately when produced
Top-p	Nucleus sampling — restricts token choice to the smallest set of tokens whose combined probability reaches p%
Top-k	Restricts token choice to only the top *k* most likely next tokens
Frequency penalty	Discourages the model from repeating the *same words/phrases*
Presence penalty	Discourages the model from repeating the *same ideas/topics*

Knowledge cutoff, again — this time from a business angle Every LLM has a "dead date" (knowledge cutoff). If you queried an old model like GPT-2 in 2026, it simply has no idea what happened after its training ended — this is exactly why models get new versions released every year: the world keeps producing new information (news, movies, events), and a static model becomes stale.

Why enterprises need RAG (the Wipro example) Scenario: A company like Wipro buys a large block of tokens (e.g., 1 million tokens) for a project's proof-of-concept, and needs to decide which model generation to use (GPT-4, GPT-5, GPT-6, etc.) — and questions arise like whether unused credit carries over.

The deeper issue: **if a company already has its own client data/documents, why would it rely on the LLM searching the open internet?** It shouldn't — client

documents should be handled by **RAG**, not raw LLM knowledge, for both accuracy and confidentiality reasons.

> **Every company that wants to let an LLM access its internal documents should implement RAG — regardless of which LLM they use.** More recent/updated LLMs generally give more accurate answers because they were trained on newer data, but RAG is what solves the "access my own private documents" problem, which no amount of newer training data can fix.

Privacy angle: LLMs don't inherently "contain" your personal files just because you asked them something — but processing large personal documents through a general-purpose model (rather than a properly scoped RAG system) raises real privacy/processing concerns.

---

6. Temperature, Top-p & Top-k in Depth (10th, continued)

Temperature

- Range: typically **0 to 2**.
- **Low temperature (near 0)** → low randomness → safe, predictable, "boring" output. Same prompt tends to give very similar answers each time.
- **High temperature (near 2)** → high randomness → more creative, more varied, but also more prone to going off-topic or making things up.
- **Default** is usually around **1** — a balance.

Example from class: ask a model to talk about an elephant and a zebra.

- At **temp 0**: you'd get plain, literal, predictable text — "elephant," "zebra."
- At **temp 2**: you might get playful/unusual output — because the model is sampling far more loosely from possible next words (e.g., "eleph-ant$$", "zebr&ra" — exaggerated to show how unpredictable it gets).

Different model types, different practical ranges:

- Text-based LLMs (Gemini, OpenAI, LLaMA, Anthropic/Claude): temperature range 0–2.
- Image-based/multimodal generation: often more constrained, roughly 0–1, since too much randomness in image generation degrades coherence faster than in text.

Top-k Limits the model to choosing its next token from only the **top k most probable candidates** (e.g., top 10, top 50, top 100). A smaller k = safer, more focused output.

Top-p (nucleus sampling) Instead of a fixed count, top-p selects from the smallest group of tokens whose *cumulative probability* reaches a threshold p (e.g., 95%). So if

top-p = 0.95, the model considers only the tokens that together make up 95% of the likely probability mass — cutting off the long tail of unlikely tokens.

Why they're used together Temperature, top-p, and top-k are often tuned together to make sure the model consistently produces **coherent** responses — output that flows naturally and is understandable to any user. If this alignment is off, you get responses that feel disjointed or nonsensical, and users lose trust in the product (a real risk for any company building on top of an LLM).

A commonly cited "balanced" starting configuration:

```
temperature = 1
top_p = 0.95
top_k = 40   (out of a possible 0-100 range)
```

Max length / max tokens Sets a hard ceiling on how much text (in tokens) the model is allowed to output in one response — e.g., setting `max_tokens = 100` means the response will be cut off at ~100 tokens even if the model "wanted" to keep going.

Stop sequence A specific marker string (like `"END"`, `"STOP"`, or a custom token) that immediately halts generation the moment it appears — useful for structured outputs where you know exactly where a response should end.

Frequency penalty (range 0–2) Reduces the chance the model repeats the exact same **words or phrases** it has already used in the response.

Presence penalty (range 0–2) Reduces the chance the model repeats the same **ideas or topics**, even if phrased differently — pushes the model toward introducing new content rather than circling the same point.

---

7. Shot-Based Prompting & Chain-of-Thought (10th–11th)

Shot-based learning/prompting

This describes **how many examples** you give the model before asking it to perform a task.

- **Zero-shot**: no examples given at all — the model must rely purely on what it already knows.
  - Analogy: a 1-year-old who has *never seen* a zebra is asked to draw one. Without any reference, the drawing will likely be inaccurate — the model/child has "zero knowledge" to draw from.
- **One-shot**: exactly **one** example is given before the task.
  - Analogy: show the same child **one** picture of a zebra, then ask them to draw one — their attempt will be closer, but still limited.

- **Few-shot**: **multiple** examples are given before the task.
    - Analogy: show the child several zebra pictures — their drawing will likely be much more accurate, since they've seen more variation.

> **Important fact from class**: by default, when you type a normal prompt into any LLM chat interface without giving examples, that is **zero-shot** prompting — you get whatever the model already "knows," with no guiding examples.

Chain-of-Thought (CoT) prompting

Instead of asking a model to jump straight to a final answer, CoT prompting asks it to **reason step-by-step** before answering — this dramatically improves accuracy on math, logic, and multi-step problems.

- **0-CoT**: the model jumps straight to the answer with no shown reasoning. Example: `24 / 6 = 4` — answer given directly, no steps shown.
- **1-CoT**: one worked example of step-by-step reasoning is shown before the actual question.
- **Few-CoT**: multiple worked examples of step-by-step reasoning are shown — generally gives the **highest accuracy** among the three, since the model has the most guidance on *how* to reason, not just *what* the answer looks like.

Example of a "step-by-step" reasoning breakdown for `24 / 6`:

> "24 chocolates need to be split into 6 equal groups. Give 1 to each group, repeat until all 24 are distributed — you find each group ends up with 4." That full walk-through is what CoT reasoning looks like, versus just stating "4."

Related idea from your notes: **thinking level** (high/low) — some modern models let you configure how much internal step-by-step reasoning they do before responding (more reasoning = generally more accurate but slower/more expensive).

Self-Consistency prompting (11th)

Instead of asking the model once, you ask it to solve the **same problem multiple times** (e.g., 3 times), possibly with slightly varied phrasing, and then take the answer that appears most consistently across those attempts. This mirrors the idea of running an algorithm (like SVM) multiple times and checking whether you get the same result — consistency across multiple attempts increases confidence in the answer.

Out-of-date knowledge in prompting

Directly tied to knowledge cutoff (Topic 3): if you ask a model something requiring current information — e.g., "what are the new Bollywood releases in 2026?" — and its training data ends earlier, it cannot answer accurately without external tools (web search or RAG).

Role-playing prompts

Instructing the model to respond **as if it were a specific persona or role** (e.g., "act as a senior Python developer," "respond as a strict interviewer") — this shapes tone, vocabulary, and the type of reasoning the model applies.

ReAct (Reason + Act)

A prompting pattern where the model **alternates between reasoning and taking an action** (like calling a tool, searching, or performing a calculation) rather than just reasoning in isolation — this is foundational to how modern "agentic" LLM systems work (the model reasons → decides it needs a tool → acts → observes result → reasons again).

What is a "reasoning model"?

A model specifically designed/trained to show or perform deeper step-by-step reasoning before answering, rather than pattern-matching straight to a response. Class example: ask an LLM the distance between the sun and the moon — if the model doesn't reason carefully (or lacks grounding), it can produce a confidently wrong answer. This is a core hallucination risk: **fluent-sounding ≠ accurate**.

Building prompts with plain Python (before touching real LLM APIs)

Before calling any actual LLM, the class practiced constructing prompts programmatically:

1. Using `print()` to display prompt text.
2. Using `input()` to take user input dynamically.
3. Using loops to generate repeated/varied prompt structures.
4. Generating prompts **dynamically** (e.g., inserting variables into a template string).
5. Then moving into more advanced prompt engineering techniques once the basic mechanics were clear.

---

8. Claude Product Family Overview

Claude (by Anthropic) is one LLM with several different **interfaces/products** built around it:

- **Claude Chat** — the standard conversational interface.
- **Claude Code** — an agentic coding assistant for writing/debugging/reviewing code.
- **Claude Cowork** — a workspace tool designed for multiple people/tasks to collaborate using Claude on shared work.
- **Claude Design** — for design-oriented tasks.
- Available via **Claude Web** and **Claude Desktop**.

Important positioning point from class: Claude is considered very strong specifically as an **LLM (text/code)** — it is *not* primarily a multimodal image/video generation model. If you want strong text generation, reasoning, or coding help, Claude is a strong choice; if you want image/video generation, other multimodal models (like Gemini) are more suited.

---

9. Running Claude via Ollama & Claude Cowork in Practice (15th)

Claude is not open-source — using it normally requires paying for API/subscription access.

Using Claude "freely" via Ollama: Ollama lets you run **open-source** models (like Gemma) locally on your own machine, at no per-call cost — this was presented as an alternative path when you don't want to pay for Claude's hosted API, though it's a different (open) model, not literally "free Claude."

```
ollama launch --model gemma
```

If you do want to use Claude's actual hosted service, you'll eventually need to set up billing.

Ways to access Claude:

1. Claude Chat (web)
2. Claude Cowork
3. Claude Desktop

Claude Cowork — practical example (dashboard project) Imagine a task requiring **4 different roles** working in parallel:

1. Python code to extract numbers/data from a website.
2. Power BI — connecting cleaned data for visualization.
3. Text cleaning.
4. Number validation.

Normally, this would need **4 separate employees** working together. Claude Cowork is designed to let **one person** orchestrate this kind of multi-step, multi-skill workflow — effectively doing the coordinated work of a small team from a single project space.

Key takeaways:

1. How to run Claude via Ollama (as an open-source-friendly alternative).
2. What Claude Cowork does (coordinating work that would otherwise need multiple specialists).

3. How to generate a Claude API key from `platform.claude.com`.

---

10. FastAPI & the Claude Model Lineup (16th)

What is FastAPI? A Python web framework that lets your application **communicate** with websites, mobile apps, frontend apps, ML models, and other AI/LLM applications — essentially the "glue" that exposes your Python logic as an API that other systems can call.

Name breakdown: FastAPI = **Fast** + **API** — built to be fast and easy to build APIs with.

Practical example — a doctor's office app A doctor uses a tablet to record patient visits: name, phone, age, symptoms. Rather than buying and maintaining a full database server, the clinic can use **FastAPI to act like a lightweight backend**, storing each patient's data (e.g., as JSON), without the overhead of standing up a full DB infrastructure. This mirrors a common real business pattern: many companies prefer using FastAPI-based lightweight services over paying for and maintaining full database/server setups.

Supporting tools:

- **Uvicorn** — the server that actually *runs* your FastAPI application.
- **Pydantic** — used to validate and structure the data your API receives/sends (ensuring, e.g., "age" is actually a number, "phone" is a valid string format, etc.).
- **Swagger UI** — an automatically generated interactive documentation page for your API, letting you test endpoints directly in the browser.

FastAPI + Claude: a common real pattern — expose a Claude-powered feature (e.g., a chatbot or document analyzer) through a FastAPI backend, so other applications (web, mobile) can call it as a standard API.

Claude model lineup mentioned in class (roles, not fixed forever):

- **Sonnet 5** — general everyday-task model.
- **Haiku 4.5** — fastest, for quick/simple answers.
- **Opus 5** — for complex, heavier tasks.
- **Fable 5.1** — positioned for the toughest challenges.

(As with all model names/tiers, treat this as a snapshot — providers frequently rename and re-tier their model lineups.)

LangGraph vs LangChain

- **LangChain** — fundamentally **chain-based**: you link steps together linearly (model → prompt → output, and so on).

- **LangGraph** — fundamentally **graph-based**: you define **nodes** (functions/steps) and **edges** (the transitions/data flow between nodes), allowing much more complex, non-linear, branching, or looping workflows — this is the foundation for building **agents**.

---

11. LangChain Academy, Indexing & Intro to RAG (17th)

Gen AI projects: Java or Python? Generative AI development can technically be done in either language — Python is by far the dominant ecosystem (most libraries, tutorials, and community support), but Java-based Gen AI development is also possible for teams already invested in Java infrastructure.

LangChain Academy — a learning resource associated with **Harrison Chase** (LangChain's creator), useful as a deeper self-study reference beyond the class.

Types of indexing (for retrieval systems):

- **Vector index** — indexes data by semantic meaning (embeddings) for similarity search.
- **Metadata index** — indexes structured attributes (e.g., date, author, category) to allow filtering.
- **Full-text index** — traditional keyword-based search (like searching for exact words, similar to a search engine).

Memory — the ability for an application (especially agent-based systems in LangGraph) to remember previous interactions/context across turns, rather than treating each request in isolation.

Introduction to RAG (Retrieval-Augmented Generation)

The central business problem RAG solves:

- A general LLM (like ChatGPT) answers well from its *general* training knowledge.
- But it **cannot access your company's or client's specific/private files** unless you explicitly give it that data.
- **Should you upload confidential client data directly into a public chatbot like ChatGPT?** Generally, **no** — this raises serious data privacy and contractual (MOU) concerns for most organizations.

This is exactly why companies build their **own RAG systems** instead of relying purely on a hosted chatbot: it lets you combine the LLM's language ability with your own **private, controlled data**, without exposing that data externally.

---

12. LLM vs RAG & the RAG Ecosystem (18th)

LLM vs RAG, recap and reinforcement

- **LLM alone**: pulls from what it learned during training (general internet-scale knowledge) → higher hallucination risk, especially for anything specific, recent, or private.
- **RAG**: pulls from a **vector database** you control and populate → much lower hallucination risk, because answers are grounded in retrieved, verifiable source documents.

Knowledge cutoff example (concrete) If GPT-3's knowledge cutoff is 2023, it simply cannot correctly answer "what movies came out in September 2026?" — it only "understands" the world as it existed up to 2023.

Cost/token angle Organizations that purchase LLM access (e.g., paying for GPT-3 tokens) need to plan around the fact that:

1. Tokens cost money and get consumed with every request/response.
2. A model's training cutoff limits what it can reliably answer without external augmentation (again pointing back to RAG).

RAG Ecosystem — the four flavors

Type	Description
Simple RAG	User asks a text question → system retrieves relevant text chunks → generates a text answer. The most common and easiest to build.
Multi-RAG	Can handle multiple modalities — e.g., retrieving information from an image based on a text query, or pulling text from a document/URL.
Vector RAG	Retrieval based on vector embeddings and similarity search (the "standard" RAG approach).
Vectorless RAG	Retrieval without relying on vector embeddings — e.g., structured/keyword-based retrieval methods instead.

> Multi-RAG is noted as significantly **harder to build** well — tools like **NotebookLM** exist precisely because building robust multimodal retrieval yourself is a substantial engineering challenge.

---

13. RAG at Scale, Vector Databases & Local LLM Setup (19th)

Handling massive document volumes If you're working with billions or trillions of documents, you can't just keep everything in memory — you need proper **database**

infrastructure built for scale (this is where "big data" handling practices come in).

Vector databases used for RAG:

- **ChromaDB**
- **FAISS** (Facebook AI Similarity Search)

Both let you store embeddings and perform fast similarity search across huge collections of vectors.

Embeddings — open source vs proprietary

- **Hugging Face** offers many **open-source embedding models** you can use for free/self-hosted.
- Many LLM providers also offer their **own proprietary embedding models** as part of their platform.

Model specialization map (as discussed in class):

Strength	Model(s)
Text-based generation	OpenAI, Claude
Multimodal	Gemini
RAG-friendly	Claude
Coding	Claude Code
Image generation / robotics	Gemini

Simple RAG vs Multi RAG, reinforced

- Simple RAG: user enters text → system retrieves text.
- Multi RAG: user enters text → system retrieves an *image* (or other modality) — much harder, active research area. This is a major reason tools like **NotebookLM** exist — to abstract away that complexity for end users.

Gen AI vs Agentic AI — the industry timeline

- **2023** — the big Generative AI boom (ChatGPT-driven).
- **2025** — **Agentic AI** boom (models that can autonomously plan, use tools, and act, not just respond).
- Looking ahead (as discussed): 2023 = guardrails/security foundations; 2025 = agentic systems maturing; 2026 = LLM security becomes a major focus; 2027 (projected) = "content loop engineering" — deeper integration of generated content into ongoing pipelines/workflows.

Career framing: pay attention to how job descriptions (JDs) on LinkedIn frame Gen AI/RAG skills — industries like **banking, insurance, and healthcare** are actively building RAG-based systems around their own client data.

Practical RAG scenario: A client shares their data/documents with your company → you build a **RAG chatbot** so the client (or the client's customers) can query that data conversationally. Typically, "the code stays largely the same across projects — what changes is the PDF/data source," meaning RAG pipelines are highly reusable across different clients once built.

Class example project: An **Insurance RAG chatbot POC** — same pattern, applied to insurance documents.

Setting up local LLMs with Docker (step-by-step, as covered in class)

1. Install/update WSL (Windows Subsystem for Linux):

```
wsl --install
wsl --update
wsl --version
```

2. Install **Docker Desktop**.
3. Open Docker Desktop → Settings → Extensions → enable Docker's model/extension support.
4. Verify Docker & model support:

```
docker model help
docker --version
```

5. Find a model on Docker Hub (e.g., `ai/llama3.2`).
6. Pull the model:

```
docker model pull ai/llama3.2:1B-Q4_0
```

7. Run the model:

```
docker model run ai/llama3.2:1B-Q4_0
```

8. Test the running LLM inside Docker.
9. Continue testing/building your project from VS Code's terminal.

---

14. LLaMA Model Versions & Access (21st)

LLaMA version history (as of class):

- LLaMA 3

    - 3.1 — deprecated
    - 3.2 — deprecated
    - 3.3 — current at time of class
- LLaMA 4
    - **Scout** model
    - **Maverick** model

Can you download Meta's LLaMA directly from Meta's own platform? No — because running these models requires substantial infrastructure (large amounts of RAM, e.g., 100GB+ for bigger versions), which isn't something Meta hands out as a simple direct download experience.

How to actually get access:

1. Go to **Hugging Face** — this is the standard place to download LLaMA model weights.
2. **Send a request to Meta** to get access — Meta gates access to LLaMA weights and approves requests (this can take time).
3. You need to request access **per version** you want (e.g., separately for 3.1, 3.2, 3.3, and 4).
4. Practical workflow: connect a GPU (e.g., via Google Drive/Colab), submit the Hugging Face access request form, generate a Hugging Face API key, and configure the correct permissions once approved.

---

15. Fine-Tuning LLMs — Methods, LoRA & Quantization (22nd)

Groq — a platform/hardware+API provider known for extremely fast LLM inference, often used as an alternative way to access open models (like LLaMA) without hosting them yourself.

What is fine-tuning? Taking a general, pre-trained LLM and further training it on **your own specific dataset**, so it becomes better/specialized at your particular task or domain. If a client wants deeper insight from their own data using an LLM, applying/training a model on that specific dataset is what's meant by fine-tuning.

Which models can you fine-tune?

- **OpenAI models** — cannot be customized/fine-tuned by end users (closed weights).
- **Claude** — cannot be customized/fine-tuned by end users (closed weights).
- **Open-weight models** (LLaMA, Mistral, etc.) — **can** be fine-tuned, since you have access to the actual model weights.

The core challenge: LLMs have enormous numbers of parameters (millions to trillions). Fully retraining all of them:

- Takes enormous compute time.
- Regular CPUs **cannot handle it**; even GPUs can crash under the load.

This challenge is exactly why **Quantization** and parameter-efficient techniques were introduced.

Fine-Tuning Methods

1. FPFT — Full Parameter Fine-Tuning You retrain *every single parameter* in the model on your new dataset. Most accurate in theory, but extremely resource-intensive — often impractical for very large models on normal hardware.

2. PEFT — Parameter-Efficient Fine-Tuning Instead of touching all parameters, you only train a small additional set of parameters, leaving the original model mostly frozen. Two key PEFT techniques:

- **LoRA (Low-Rank Adaptation)**: adds small, trainable "low-rank" matrices into the network, and *only* trains those new matrices — the original (huge) model weights stay frozen. This massively reduces the number of parameters you actually need to train.

  > Example from class: a LLaMA model with ~1 trillion parameters, when using LoRA, might only need to train on the order of a much smaller number of effective parameters — meaning your system is far less likely to run out of memory or crash, and training becomes practically feasible.

- **QLoRA (Quantized LoRA)**: combines **quantization** + **LoRA**. Not only do you train fewer parameters (via LoRA), you also **reduce the numeric precision** of the model's weights (e.g., from 64-bit down to 8-bit or 4-bit), cutting memory requirements dramatically while still fine-tuning effectively.

What is Quantization? Reducing the number of bits used to represent each model weight (e.g., 64-bit → 32-bit → 8-bit → 4-bit). Lower precision = much less memory and faster computation, at the cost of some (usually small) accuracy loss. Related tools/libraries:

- **AutoGPTQ**
- **bitsandbytes**
- **Accelerate** (often paired with `bitsandbytes`)

Example LoRA configuration:

```
lora_config = LoraConfig(
    r=8,                                    # rank of the low-rank matrice
    lora_alpha=8,                           # scaling factor (alpha/r = 8/
```

```
    target_modules=["q_proj", "v_proj"],   # which layers to apply LoRA t
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)
```

- `r` **(rank)**: controls the size of the added low-rank matrices — higher rank = more trainable capacity, but more memory/compute.
- `target_modules`: which specific parts of the model architecture (e.g., attention projection layers) get the LoRA adaptation.
- `lora_dropout`: a regularization technique — randomly "drops" some connections during training to **reduce overfitting**, which in turn tends to **improve the model's real-world accuracy** (prevents it from just memorizing the training set).

Grounding (tying back to Topic 12): giving the LLM factual, verified information to base its answer on — this is the *conceptual* link between fine-tuning/RAG and reducing hallucination.

Practical challenges in fine-tuning:

- Large datasets (e.g., the IMDB dataset) can take a very long time to process/train on.
- If your GPU keeps crashing, the practical fix is usually to **start with a smaller dataset** rather than skipping the exercise entirely — get the pipeline working end-to-end first, then scale up.
- **Hugging Face** is the go-to hub for both open datasets and open models to practice fine-tuning on.

---

16. DeepSeek, Kimi, OpenRouter & Multi-Provider Access (23rd)

GPU crash troubleshooting (recap) If your GPU keeps crashing (e.g., on a dataset like IMDB because it's simply too large for your hardware), the practical solution is: **use a smaller dataset first**, get everything working, then scale — don't skip the exercise, just right-size it.

DeepSeek

- A Chinese-developed LLM (background note from class: founded by a quant/stock-market background developer, reportedly initially built for internal financial/stock modeling purposes before becoming a broader LLM).
- You **cannot download DeepSeek directly** from a simple official channel the way you might expect — instead, you access it through intermediaries:
  - **Ollama** (run it locally)

- **Groq** (fast hosted inference)
- **Docker** (containerized model running)
- **Hugging Face** (download/host it yourself)

Running DeepSeek locally via Ollama:

```
ollama run deepseek-r1
```

Class example: building a chatbot using **Ollama + Streamlit + DeepSeek-R1 (1.5B parameter version)** — a lightweight enough setup to run on a personal machine (model file size examples mentioned: ~5.2GB for a larger variant, ~1.5GB for a smaller one).

Kimi — an LLM from **Moonshot AI**, another notable non-US model provider mentioned in the "who else is building LLMs" discussion (Kimi K2 specifically referenced elsewhere in your notes).

The pricing problem & OpenRouter Increasingly, most hosted LLMs require payment to access via API. **OpenRouter.ai** is presented as a solution to a specific pain point: normally, **every LLM provider requires its own separate API key**. OpenRouter lets you use a **single key** to access many different LLMs (open-source and proprietary) through one unified API — simplifying key management significantly.

Where you can access various LLMs from:

- Groq
- OpenRouter
- Hugging Face
- (with providers like OpenAI, Claude, DeepSeek, Kimi, Gemini/Google DeepMind all accessible through one or more of these routes)

---

17. Google DeepMind & the Gemini Ecosystem (24th)

Google DeepMind is Google's dedicated AI **research** organization/company — Gemini and its related models come out of this research effort.

The Gemini/DeepMind model family (as covered):

- **Gemini** — the core LLM/multimodal line (e.g., Gemini 2.5 up through newer versions like 3.x mentioned in class).
- **Gemini Omni** — broader multi-sense/omni-modal capability.
- **Nano Banana** — an internal/codename-style project referenced in class for image-related generation.

- **Gemini Audio** — audio-focused capability.
- **Gemma** — Google's **open-weight** model family (distinct from the closed Gemini line) — this is what you'd realistically run locally via tools like Ollama.
- **Veo** — Google's video generation model.
- **Lyria** — Google's music/audio generation model.
- **Gemini Robotics** — applying Gemini's capabilities to robotics/physical-world tasks.
- **Genie 3** — a "world model" project (interactive generated environments).

Two capability tracks:

- **Gemini LLM** — pure language capability.
- **Gemini Multimodal** — image↔text and broader cross-modal generation.

Full Gen AI landscape recap

LLM providers: OpenAI, Claude, Gemini, Ollama (local), Docker (local), Groq, Hugging Face, OpenRouter, DeepSeek, Kimi — and the broader point that **every company/provider situation needs to be understood individually** (pricing, access model, openness, strengths).

Multimodal directions: text→image, image→text, text→audio, audio→text, text→video, video→text — an enormous and fast-moving area of active research, with new libraries and techniques being released constantly.

> Class observation: despite decades of progress in generating text and code, **image/video generation is still a comparatively newer and less mature frontier** — e.g., Claude, while extremely strong at text, still doesn't fully compete in image generation. Related computer-vision-era tools like **OpenCV, YOLO, TensorFlow, and Keras** remain relevant in this broader "generate/understand visual content" space.

---

18. Diffusion Models & Text-to-Image (25th)

Where text-to-image generation came from

- **Diffusion models** as a technique were introduced back in **2015** by the **CompVis** research group.
- In **2022**, that research group's work was **sold to Stability AI**, which used it to build and release **Stable Diffusion** — one of the most well-known text-to-image models.
- Stable Diffusion specifically uses the **Latent Diffusion** technique — instead of generating an image pixel-by-pixel in full resolution space (very expensive), it

works in a compressed "latent" representation, making generation far more efficient, then decodes that back into a full image.

Building a multimodal system without an API key The class flagged this as an advanced, longer-term project (roughly 3 weeks of build time) — involving architectures like:

- **CLIP** — connects text and images into a shared understanding space (used to guide/evaluate generation).
- **VQGAN** — a GAN-based approach for generating high-quality images, often paired with CLIP for text-guided image generation.

Practical hardware note: one class project was scoped around a **14–15GB RAM** budget — a reminder that even "build it yourself" generative projects need realistic hardware planning, not just theoretical architecture knowledge.

Closing mention: **Quantum Machine Learning** was flagged as an emerging adjacent field to keep an eye on, though not covered in depth in this course.

Quick-Reference Glossary

Term	Meaning
LLM	Large Language Model — text-focused generative model
SOTA	State-of-the-Art — typically refers to top multimodal models
GAN	Generative Adversarial Network (Generator vs Discriminator)
VAE	Variational Autoencoder (Encoder → Decoder)
RAG	Retrieval-Augmented Generation — grounding answers in retrieved documents
Token	A chunk of text (word/sub-word) the model processes
Embedding	A numeric vector representing meaning
Attention	Mechanism for weighing relevance between words
Q/K/V	Query/Key/Value — the internal mechanics of attention
Context length	Max amount of text a model can process at once
Knowledge cutoff	Date beyond which a model has no training data

Term	Meaning
Fine-tuning	Retraining a model on your own custom dataset
LoRA / QLoRA	Efficient fine-tuning techniques requiring far less compute
Quantization	Reducing numeric precision of model weights to save memory
Temperature / Top-p / Top-k	Parameters controlling randomness/creativity of output
Zero/One/Few-shot	How many examples you give a model before asking a task
Chain-of-Thought (CoT)	Prompting the model to reason step-by-step
ReAct	Reason + Act — the basis of agentic LLM behavior
LangChain / LangGraph	Frameworks for chaining LLM calls / building agent graphs
LangSmith	Monitoring/observability tool for LLM apps

End of notes. Ready to go deeper on any specific section, run practice questions, or work through the code examples hands-on — just point to a topic number.